



Bricks

Image Classification

UNO

Q

การจำแนกภาพ

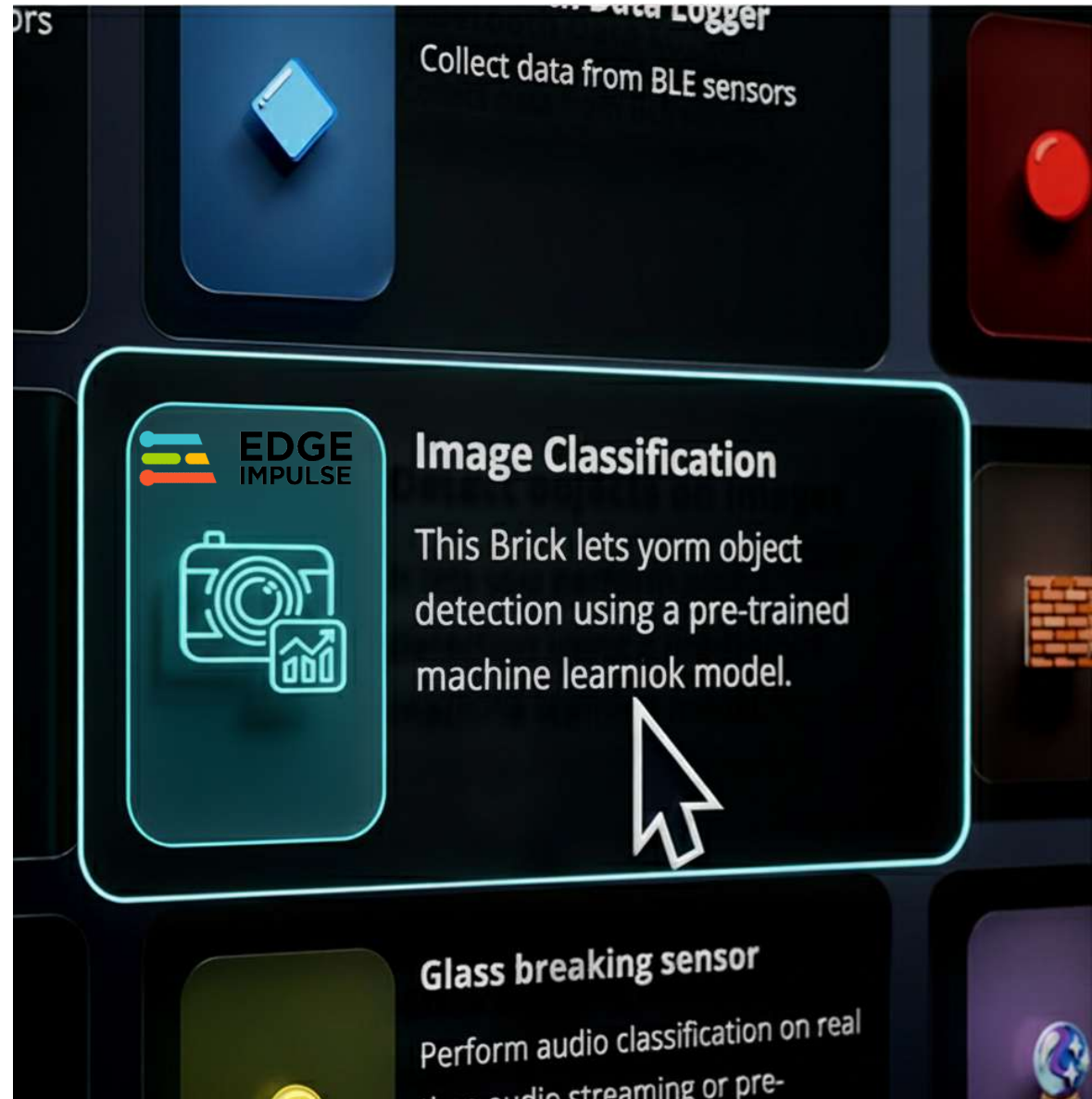


Image Classification Brick และ Object Detection Brick บน Arduino App Lab แตกต่างกันอย่างไ?

Object Detection

- ใช้เพื่อหาวัตถุหลาย ๆ ชนิดในภาพเดียว
- ให้ทั้ง คลาสของวัตถุ และ ตำแหน่ง (bounding box) ว่า วัตถุอยู่ตรงไหน
- เหมาะกับงานที่ต้องรู้ว่า “มีอะไร” และ “อยู่ตรงไหน”

ตัวอย่างงานเหมาะกับ Image Classification

- ตรวจสอบคนหลายคนในภาพ
- ตรวจสอบจํานวนรถในลานจอด
- ตรวจสอบจับกล่องสินค้าในคลัง
- ระบบนับวัตถุ (counting)

```
[1] Object: person, Confidence: 0.86
Bounding box: [x1, y1, x2, y2]

[2] Object: car, Confidence: 0.74
Bounding box: [x1, y1, x2, y2]
```

Image Classification

- Image Classification ใช้โมเดล MobileNet V2 ที่ถูกพรีเทรนบนชุดข้อมูล ImageNet (1,000 class)
- ใช้เพื่อหาวัตถุหลาย ๆ ชนิดในภาพเดียว
- ใช้เพื่อบอกว่า “ภาพนี้คืออะไร” หรือ จัดประเภทของภาพทั้งใบ
- ไม่มีตำแหน่งวัตถุ (ไม่รู้ว่าจะอยู่ตรงไหนในภาพ)
- รับอินพุตเป็นภาพทั้งภาพ แล้วส่งผลลัพธ์เป็น คลาส + ค่าความมั่นใจ

ตัวอย่างงานเหมาะกับ Image Classification

- ตรวจสอบว่าเป็น แมว หรือ หมา
- แยกว่าภาพคือ สมุด / ปากกา / กรรไกร
- จำแนกว่าคือ พืชผลไม้ / พืชผัก / พลาสติก

```
Class: "cat"
Confidence: 0.92
```

รายการ	Object Detection	Image Classification
บอกชนิดวัตถุ	ทำได้	ทำได้
ระบุตำแหน่งวัตถุ	ทำได้โดยมี Bounding Box ให้	ทำไม่ได้
ตรวจหลายวัตถุในภาพเดียว	ทำได้	ทำได้
ระดับความยาก/ง่าย	พริก 3 เมล็ด	พริก 1 เมล็ด
ใช้ทรัพยากร	หนัก	เบา
ความเร็ว	ช้า	เร็ว
แม่นยำถ้ามีวัตถุเดียว	พริก 1 เมล็ด	พริก 3 เมล็ด
โมเดลที่ใช้	YOLOX	Edge Impulse

Image Classification มักให้ความแม่นยำสูงกว่า Object Detection เพราะไม่ต้องแบ่งวัตถุเป็นกรอบ ๆ

Image Classification ต้องการ dataset ที่เรียบง่ายขึ้นกว่า ไม่ต้องทำ annotation (ลากกรอบวัตถุเหมือน Object Detection) → ประหยัดเวลาเทรนมาก



Brick “Object Detection” ของ Arduino App Lab ที่ทำงานร่วมกับ Edge Impulse ใช้โมเดลตระกูล YOLOX แบบเวอร์ชัน Tiny โดยผ่านการ optimize ที่ถูกแปลงเป็น Edge Impulse EIM model (Quantized) โมเดลนี้ถูกลดขนาดอย่างหนักเพื่อให้รันได้บนอุปกรณ์ Edge

****หมายเหตุ**** Object Detection Brick ใช้โมเดลแบบ YOLOX ผ่าน Edge Impulse แพลตฟอร์ม โดยอัปโหลดภาพขึ้น Edge Impulse Studio เทรนด้วย YOLOX และ Deploy ลง Arduino UNO Q

```
from arduino.app_bricks.image_classification import ImageClassification
image_classification = ImageClassification()
# out = image_classification.classify(frame, image_type="png", confidence=0.35)
out = image_classification.classify(frame)

for i, obj_det in enumerate(out["classification"]):
    detected_object = obj_det.get("class_name", None)
    confidence = obj_det.get("confidence", None)
    print(f"[{i+1}] Object: {detected_object}, Confidence: {confidence}")
```

หากจะใช้คำสั่งคัดลอกไฟล์ที่นามสกุล *.jpg และ *.png จากโฟลเดอร์หนึ่งไปยังอีกโฟลเดอร์หนึ่ง
บนลินุกส์จะใช้คำสั่งอะไร

/home/arduino/ArduinoApps/object-detection-brick-testing ต้นทาง

/home/arduino/ArduinoApps/image-classification-brick-testing ปลายทาง

```
cp /home/arduino/ArduinoApps/object-detection-brick-testing/*.jpg  
/home/arduino/ArduinoApps/object-detection-brick-testing/*.png  
/home/arduino/ArduinoApps/image-classification-brick-testing
```

ตรวจสอบไฟล์โฟลเดอร์ปลายทาง

```
ls /home/arduino/ArduinoApps/image-classification-brick-testing
```

```
arduino@UNOQThana:~/ArduinoApps$ ls /home/arduino/ArduinoApps/image-classification-brick-testing  
app.yaml  Car.jpg  Cat2.png  Cat.jpg  Dog2.png  Dog.jpg  Man.jpg  Pickup.jpg  python  README.md  sketch  Woman.jpg
```

▼ Files

- ▶ python
- ▶ sketch
- Car.jpg
- Cat.jpg
- Cat2.png
- Dog.jpg
- Dog2.png
- Man.jpg
- Pickup.jpg
- README.md
- Woman.jpg
- app.yaml

ในการเรียกใช้ Brick ในครั้งแรก
อาจจะจำเป็นต้องโหลดไฟล์ต่าง ๆ
เข้ามาไว้ที่บอร์ด Arduino UNO
Q (ใช้เวลานานพอสมควร)

```
10f70eb0f207 Downloading [=====] 609B/609B
5bf3a062ef59 Downloading [=====] 609B/609B
5bf3a062ef59 Verifying Checksum
5bf3a062ef59 Download complete
45d6f305a55f Download complete
816c40ead22d Downloading [>] 310.1kB/30.14MB
816c40ead22d Downloading [=>] 621.4kB/30.14MB
816c40ead22d Downloading [==>] 932.7kB/30.14MB
816c40ead22d Downloading [===>] 1.244MB/30.14MB
816c40ead22d Downloading [====>] 1.555MB/30.14MB
816c40ead22d Downloading [=====] 1.867MB/30.14MB
816c40ead22d Downloading [=====] 2.178MB/30.14MB
816c40ead22d Downloading [=====] 2.8MB/30.14MB
cd0b669ead3f Downloading [=====] 1.378kB/2.787kB
cd0b669ead3f Downloading [=====] 2.787kB/2.787kB
cd0b669ead3f Download complete
816c40ead22d Downloading [=====] 3.112MB/30.14MB
816c40ead22d Downloading [=====] 3.423MB/30.14MB
a319d904a289 Downloading [=====] 809B/809B
a319d904a289 Verifying Checksum
a319d904a289 Download complete
816c40ead22d Downloading [=====] 3.734MB/30.14MB
816c40ead22d Downloading [=====] 4.046MB/30.14MB
816c40ead22d Downloading [=====] 4.357MB/30.14MB
816c40ead22d Downloading [=====] 4.668MB/30.14MB
53ccb214417f Downloading [>] 506.7kB/49.2MB
816c40ead22d Downloading [=====] 4.98MB/30.14MB
3999456cf735 Downloading [>] 539.5kB/70.15MB
816c40ead22d Downloading [=====] 5.291MB/30.14MB
53ccb214417f Downloading [=>] 1.015MB/49.2MB
816c40ead22d Downloading [=====] 5.602MB/30.14MB
53ccb214417f Downloading [=>] 1.523MB/49.2MB
816c40ead22d Downloading [=====] 5.913MB/30.14MB
3999456cf735 Downloading [>] 1.08MB/70.15MB
816c40ead22d Downloading [=====] 6.225MB/30.14MB
53ccb214417f Downloading [==>] 2.03MB/49.2MB
816c40ead22d Downloading [=====] 6.536MB/30.14MB
816c40ead22d Downloading [=====] 6.847MB/30.14MB
3999456cf735 Downloading [=>] 1.621MB/70.15MB
816c40ead22d Downloading [=====] 7.159MB/30.14MB
53ccb214417f Downloading [==>] 2.523MB/49.2MB
816c40ead22d Downloading [=====] 7.47MB/30.14MB
```